

Hashing – Short Revision Notes (Quick + Clear)

1. What is Hashing?

Hashing is a technique, not a data structure.

It is used to **store and access data fast** by mapping a **key** to an **index** using a **hash function**.

Idea:

key → hash function → index → data

2. Hash Table

A **hash table** is the data structure that implements hashing.

Internally it has:

- An array (buckets)
- A hash function
- A collision handling method

C++ STL structures using hash tables:

- `unordered_map`
- `unordered_set`

3. Hash Function (Division Rule)

Most basic hash function:

$\text{hash}(\text{key}) = \text{key} \% \text{table_size}$

Example:

$17 \% 7 = 3$

So key 17 goes to index 3.

4. Collision

A **collision** happens when two keys map to the same index.

Example:

$10 \% 7 = 3$

$17 \% 7 = 3$

Collisions are **unavoidable**.

5. Collision Handling (Concept)

Separate Chaining

- Each index stores a list

index 3 → (10,100) → (17,200)

This is the most common idea.

6. Load Factor

load factor = elements / table size

- If load factor increases → collisions increase
- When it becomes large → **rehashing** is done

7. Time Complexity

Operation	Average	Worst
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

8. Direct Addressing vs Hashing

Direct Addressing (Array)

```
int freq[100] = {0};
```

- Index = key
- No collision
- Works only for small range

Hashing

- Index = hash(key)
- Collisions possible
- Works for large values

9. Hashing in C++ STL

unordered_set (Only Keys)

```
unordered_set<int> s;
```

```
s.insert(x);
```

Use when:

- Only **existence check** is needed

Check existence:

```
s.count(x); // 0 or 1
```

unordered_map (Key-Value)

```
unordered_map<int,int> mp;
```

```
mp[x]++;
```

Use when:

- **Frequency / count** is needed

10. Choosing Between Set and Map (IMPORTANT)

Requirement	Use
Only check if element exists	unordered_set
Count frequency	unordered_map
First repeating / non-repeating	unordered_map

11. Solved Example – First Repeating Element

Problem

Find the **first element that appears more than once**.

```
arr = {10, 5, 3, 4, 3, 5, 6}
```

Output:

5

Approach (Hashing Only)

1. Store frequency of each element using unordered_map
 2. Traverse array again
 3. First element with frequency > 1 is the answer
-

Code

```
#include <iostream>
#include <unordered_map>
using namespace std;

int firstRepeating(int arr[], int n) {
    unordered_map<int,int> mp;

    // Step 1: Frequency count
    for (int i = 0; i < n; i++) {
        mp[arr[i]]++;
    }

    // Step 2: Find first repeating
    for (int i = 0; i < n; i++) {
        if (mp[arr[i]] > 1) {
            return arr[i];
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 5, 3, 4, 3, 5, 6};
    cout << firstRepeating(arr, 7);
}
```

12. Common Mistakes

- Using unordered_set for frequency problems ❌
- Forgetting that unordered_set::count() only checks existence
- Expecting ordered output from unordered_map

13. One-Line Revision Summary

- Hashing is a **technique**
- Hash table is a **data structure**
- unordered_set → existence
- unordered_map → frequency

- First repeating / non-repeating → **map + array traversal**

END OF REVISION NOTES